

SCD Pipeline — Complete Step-by-Step Guide

A full walkthrough of building this project end to end: AWS infrastructure (Terraform) → Docker/Airflow on EC2 → S3 → Snowflake ingestion → SCD Type 1 → SCD Type 2. Every error hit along the way is documented with its cause and fix, in the order they actually happened.

Contents:

- [Part 1: AWS Infrastructure \(Terraform\)](#)
 - [Part 2: Docker & Airflow on EC2](#)
 - [Part 3: Snowflake Setup & SCD Pipeline](#)
 - [Full Lessons-Learned Summary](#)
 - [Cleanup](#)
-

Part 1: AWS Infrastructure (Terraform)

What this provisions

- **S3 bucket** — storage for raw CSVs before they reach Snowflake
- **EC2 instance** — compute server to run Docker/Airflow
- **IAM role + instance profile** — lets the EC2 instance access S3 without hardcoded credentials
- **Security group** — firewall rules for SSH (22) and port-forwarding (4000–38888)
- **SSH key pair** — for secure login to the EC2 instance

Files: `providers.tf`, `main.tf`, `outputs.tf`.

Step 1.1 — Install Terraform

Error: Double-clicking `terraform.exe` did nothing.

Cause: Terraform is a **command-line tool** — it has no GUI window to open.

Fix:

1. Move `terraform.exe` to a permanent folder (e.g. `C:\terraform`).
2. Add that folder to the Windows `PATH` environment variable.
3. Open a **new** terminal (must be opened after editing `PATH`).
4. Verify: `terraform -version`

Step 1.2 — Empty project folder

Error:

Error: No configuration files

happened when running `terraform plan` right after `terraform init` in a folder with no `.tf` files.

Cause: `terraform init` only sets up the working directory (providers/backend) — it doesn't require actual infrastructure code. `plan/apply` need real `.tf` files.

Fix: Add `main.tf`, `providers.tf`, `outputs.tf` to the folder, then re-run `terraform init`.

Step 1.3 — Install & configure AWS CLI

Terraform's AWS provider needs valid credentials (`provider "aws" { profile = "default" }`).

Steps:

1. Generate an AWS Access Key ID + Secret Access Key (AWS Console → Security credentials).
2. Install AWS CLI v2 for Windows.
3. Run `aws configure` → enter Access Key ID, Secret Access Key, region (`us-west-2`), output format (`json`).

Verify with:

```
aws sts get-caller-identity
```

4.

Note: The ARN returned showed `:root` — meaning root account credentials were used, not a dedicated IAM user. Fine for a course project, but a dedicated IAM user with `AdministratorAccess` is better practice long-term.

Step 1.4 — Choosing a free-tier EC2 instance type

Initial config: `instance_type = "t2.large"` — not free tier, ~\$0.08/hr.

Correction during the process: Initially assumed only `t2.micro/t3.micro` were free-tier eligible. However, the AWS Console's instance-type picker showed `c7i-flex.large` tagged "Free tier eligible" — this account was on a newer 6-month free-tier offer (AWS updated free-tier structure in 2024/2025) that includes broader instance types than the classic micro-only model.

Fix: Updated `main.tf`:

```
instance_type = "c7i-flex.large" # was "t2.large"
```

Lesson: Free-tier eligibility changes over time and by account — always verify current eligibility directly in the AWS Console rather than relying on older assumptions.

Step 1.5 — First successful deploy

```
terraform init
terraform validate
terraform plan
terraform apply # type 'yes'
```

Result: `Apply complete! Resources: 9 added, 0 changed, 0 destroyed.`

Step 1.6 — SSH: identity file not found

Error:

Warning: Identity file `./ec2-scd-snowflake-us-west-2-tf.pem` not accessible: No such file or directory.

...

Permission denied (publickey,gssapi-keyex,gssapi-with-mic).

Cause: SSH was run from `C:\Users\<user>` while the `.pem` file actually lives in the Terraform project folder. SSH couldn't find the key, fell back to no auth method.

Fix: `cd` into the same folder as the `.pem` file before running SSH.

Step 1.7 — SSH: bad permissions on the private key

Error:

```
Bad permissions. Try removing permissions for user: BUILTIN\Users...  
WARNING: UNPROTECTED PRIVATE KEY FILE!  
Load key "...": bad permissions  
Permission denied
```

Cause: Windows gives new files broad default permissions readable by `BUILTIN\Users`. SSH refuses private keys that others could read.

Fix (PowerShell):

```
icacls ec2-scd-snowflake-us-west-2-tf.pem /inheritance:r  
icacls ec2-scd-snowflake-us-west-2-tf.pem /grant:r "$($env:USERNAME):(R)"
```

Fix (cmd.exe):

```
icacls ec2-scd-snowflake-us-west-2-tf.pem /grant:r "%USERNAME%:(R)"
```

Result: SSH connected successfully to Amazon Linux 2.

Step 1.8 — Re-running Terraform without duplicating resources

If a prior session left some resources (e.g. the S3 bucket) undestroyed while others were torn down, don't guess — check what Terraform still tracks:

```
terraform state list
```

Anything listed is already tracked (`plan` will show `0 to add` for it). If a resource exists in AWS but isn't in the state (state file lost), import it instead of letting `apply` recreate it:

```
terraform import aws_s3_bucket.my_bucket <bucket-name>
```

Step 1.9 — Re-apply: local .pem file access denied

Error, on a second `terraform apply` after the S3 bucket already existed:

Error: Create local file error

Original Error: open ./ec2-scd-snowflake-us-west-2-tf.pem: Access is denied.

(EC2 instance, IAM role, security group, key pair all created fine — only the local `.pem` write failed.)

Cause: The earlier fix in Step 1.7 locked the `.pem` file to **read-only**, including for the file's owner. When Terraform tried to overwrite it with a fresh key, even the owning user no longer had write permission.

Fix (cmd.exe — note the earlier PowerShell-only syntax fails silently here):

```
icacls ec2-scd-snowflake-us-west-2-tf.pem /grant:r "%USERNAME%:(F)"
```

```
terraform apply --auto-approve
```

```
icacls ec2-scd-snowflake-us-west-2-tf.pem /grant:r "%USERNAME%:(R)"
```

Extra gotcha hit here: typing `grant:r "$($env:USERNAME):(F)"` without the leading `icacls` command (accidentally split across a "run X then run Y" instruction) produced:

The filename, directory name, or volume label syntax is incorrect.

because cmd tried to execute `grant:r` as a program. **Lesson:** always include the full command (`icacls ...`) on one line — treat "then" in instructions as "next separate command," never paste partial fragments.

Step 1.10 — New EC2 instance ≠ old EC2 instance's software

After a rebuild, the new instance's public IP differs, and **none of the previous session's installed software carries over** — Terraform only manages infrastructure, not what's installed on the box. SSHing in and running `docker ps` returned:

```
-bash: docker: command not found
```

This is expected on any freshly created instance — proceed to Part 2 to reinstall Docker/Airflow.

Part 2: Docker & Airflow on EC2

Step 2.1 — Accidentally running SSH from inside the remote session

Error:

```
[ec2-user@ip-172-31-50-72 ~]$ ssh -i "terraform\ec2-scd-snowflake-us-west-2-tf.pem"
ec2-user@18.246.248.208
Warning: Identity file terraform\ec2-scd-snowflake-us-west-2-tf.pem not accessible: No such file
or directory.
...
Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

Cause: The command was typed **while already logged into the EC2 instance** (note the `[ec2-user@...]` prompt), not from a fresh local Windows terminal. The Windows-style path doesn't exist on the Linux box, and this was effectively trying to SSH from the instance back to itself.

Fix: Open a genuinely new terminal window on the **local Windows machine**, `cd` to the project folder there, and run the SSH command from that separate window.

Step 2.2 — Install Docker on Amazon Linux 2

```
sudo yum update -y
sudo amazon-linux-extras install docker -y
sudo service docker start
sudo systemctl enable docker
sudo usermod -a -G docker ec2-user
```

Note: must log out and reconnect (`exit`, then SSH back in) for the group change to take effect.

Step 2.3 — Install Docker Compose

Error encountered: pasting two commands as one line —

```
sudo curl -L
"https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname
-s)-$(uname -m)" -o /usr/local/bin/docker-compose then sudo chmod +x
/usr/local/bin/docker-compose
```

produced:

```
curl: (6) Could not resolve host: then
curl: (6) Could not resolve host: sudo
curl: (3) URL rejected: Bad hostname
```

Cause: "then" was plain English separating two *separate* commands in the instructions, not something to paste literally. curl tried to treat **then**, **sudo**, **chmod** as extra URLs.

Fix: The download itself had actually already succeeded (visible in the **100 30.8M** progress line) — just run the second command alone:

```
sudo chmod +x /usr/local/bin/docker-compose
docker-compose --version
```

Step 2.4 — Transferring project files: destination folder must exist

Error:

```
scp: realpath scd-pipeline/: No such file or directory
scp: upload "scd-pipeline/": path canonicalization failed
```

Cause: Newer **scp** (SFTP-based) won't auto-create a nested destination folder.

Fix:

```
# on the EC2 instance first:
mkdir ~/scd-pipeline
```

```
# then from the LOCAL Windows machine:
scp -i "terraform\ec2-scd-snowflake-us-west-2-tf.pem" -r . ec2-user@<ip>:~/scd-pipeline/
```

Step 2.5 — Harmless scp errors on Terraform provider binaries

Errors seen during the same transfer:

```
scp: local lstat "LICENSE.txt": No such file or directory
scp: local lstat "terraform-provider-aws_v6.62.0_x5.exe": No such file or directory
scp: failed to upload directory . to ~/scd-pipeline/
```

Cause: Terraform's `.terraform/providers/...` folder sometimes uses symlinks to a shared plugin cache; `scp -r` can't follow those the same way as regular files.

Fix / non-fix: Not actually a problem — these files aren't needed on the EC2 instance at all (Terraform is a local-only tool). Everything needed for Docker/Airflow (`docker-compose.yaml`, `.env`, `dags/`, `requirements.txt`, SQL files) copied over fine.

Security note flagged: the same transfer also copied the `.pem` private key and `terraform.tfstate` files onto the EC2 instance — not ideal, since the instance is publicly reachable. Clean up later with:

```
rm ~/scd-pipeline/ec2-scd-snowflake-us-west-2-tf.pem
rm ~/scd-pipeline/terraform.tfstate ~/scd-pipeline/terraform.tfstate.backup
```

Step 2.6 — Missing `faker` module in the DAG

Error (in Airflow UI, DAG import error):

ModuleNotFoundError: No module named 'faker'

Cause: Having `faker/boto3` listed in `requirements.txt` doesn't install anything by itself. The official Airflow docker-compose template installs extra packages via the `_PIP_ADDITIONAL_REQUIREMENTS` env var, read from `.env` at container startup — and the containers had already been started before this was set correctly.

Fix:

```
# in .env:
_PIP_ADDITIONAL_REQUIREMENTS=faker boto3
```



```
# then recreate containers (restart alone isn't enough — .env is re-read on recreate):  
docker-compose down  
docker-compose up -d
```

Wait 1–2 minutes for pip install to finish at container startup, then refresh the Airflow UI.

Step 2.7 — DAG uploading to the wrong S3 bucket

Error (Airflow task log):

```
S3UploadFailedError: Failed to upload .../customer_20260905204714.csv to  
snowflake-class-project-ingestion/...:  
An error occurred (AccessDenied) when calling the PutObject operation: Access Denied
```

Cause: The DAG's default bucket (`SCD_S3_BUCKET` env var default = `"snowflake-class-project-ingestion"`) didn't match the actual Terraform-created bucket (`s3-scd-snowflake-us-west-2-tf`). The IAM role has full access to its own bucket, but not to an unrelated/nonexistent one — hence `AccessDenied`.

Fix (quick): Edit the DAG file directly:

```
# customer_data_to_s3_dag.py  
S3_BUCKET = os.environ.get("SCD_S3_BUCKET", "s3-scd-snowflake-us-west-2-tf") # was  
"snowflake-class-project-ingestion"
```

Airflow auto-detects DAG file changes within ~30 seconds — no restart needed.

Better long-term fix: set `SCD_S3_BUCKET` in `.env` and reference it in `docker-compose.yml`'s `environment:` block instead of hardcoding it in the Python file.

Result: Data began landing in `customer_raw` via the Snowpipe successfully.

Part 3: Snowflake Setup & SCD Pipeline

Step 3.1 — Create warehouse, database, tables, and stream

Run `01_table_creation.sql` as-is. Creates:

- `COMPUTE_WH` (warehouse, auto-suspends after 120s idle)
- `scd_demo.scd2` (database/schema)
- `customer_raw` — raw landing table
- `customer` — clean current-state table
- `customer_history` — full audit trail table
- `customer_table_changes` — a **stream** on `customer`, tracking inserts/updates/deletes

No errors here.

Step 3.2 — Setting up S3 → Snowflake ingestion (storage integration)

The original script uses raw AWS keys in the stage (`credentials=(aws_key_id=... aws_secret_key=...)`). Instead, a **storage integration** was set up for better security.

Attempted first:

```
storage_aws_role_arn = 'arn:aws:iam::LKBTXDE-ED08945:role/ACCOUNTADMIN'
```

Error:

SQL compilation error: invalid value ['...'] for parameter 'storage_aws_role_arn'

Cause: This mixed up two unrelated things — `LKBTXDE-ED08945` is a **Snowflake account identifier**, and `ACCOUNTADMIN` is a **Snowflake role**. `storage_aws_role_arn` must be a real **AWS IAM role ARN** that exists in the AWS account (e.g. `arn:aws:iam::254265667532:role/snowflake_s3_role`), which doesn't exist yet the first time through.

Correct sequence:

1. Create the storage integration with a placeholder ARN first.
2. Run `desc integration s3_int;` to get Snowflake's `STORAGE_AWS_IAM_USER_ARN` and `STORAGE_AWS_EXTERNAL_ID`.
3. In AWS IAM, create a **new, dedicated role** (not the EC2 instance's role) with:
 - **Trust policy:** allows Snowflake's IAM user ARN to assume it, conditioned on the matching External ID.

- **Permissions policy:** `s3:GetObject`, `s3:PutObject`, `s3:GetObjectVersion`, `s3:DeleteObject`, `s3:DeleteObjectVersion` scoped to `arn:aws:s3:::<bucket>/*`, plus `s3:ListBucket` scoped to `arn:aws:s3:::<bucket>` (note: object actions target `/*`, `ListBucket` targets the bucket itself — different resource scopes, an easy thing to get wrong).

Update Snowflake's integration with the real role ARN:

```
alter storage integration s3_int set storage_aws_role_arn =
'arn:aws:iam::254265667532:role/snowflake_s3_role';
```

4.

Step 3.3 — File format syntax error on stage creation

Error:

```
create or replace stage customer_ext_stage
url='s3://s3-scd-snowflake-us-west-2-tf/'
storage_integration = s3_int
file_format = 'CSV'
```

SQL compilation error: File format 'CSV' does not exist or not authorized.

Cause: `file_format = 'CSV'` (quoted string, no parentheses) tells Snowflake to look up a **named file format object** called `CSV` — which doesn't exist. It does not mean "set the type to CSV."

Fix (chosen — reusable named format):

```
create or replace file format csv_format
type = 'CSV'
skip_header = 1
field_optionally_enclosed_by = '';
```

```
create or replace stage customer_ext_stage
url='s3://s3-scd-snowflake-us-west-2-tf/'
storage_integration = s3_int
file_format = csv_format;
```

(Alternative: inline format `file_format = (type = 'CSV' skip_header = 1 ...)` — also valid, just less reusable.)

Result: `list @customer_ext_stage;` successfully showed the CSV files landing from the Airflow DAG.

Step 3.4 — Creating the pipe (auto-ingest)

Since the DAG generates a new filename every run (`customer_<timestamp>.csv`), the pipe was pointed at the whole stage rather than one hardcoded filename from the original script:

```
create or replace pipe customer_s3_pipe
  auto_ingest = true
  as
  copy into customer_raw
  from @customer_ext_stage
  file_format = csv_format;
```

Verified with:

```
show pipes;
select SYSTEM$PIPE_STATUS('customer_s3_pipe');
select count(*) from customer_raw;
```

Result: data landed successfully in `customer_raw`.

Step 3.5 — SCD Type 1 (`03_scd_type-1.sql`)

Understanding the merge condition — a point of confusion:

```
when matched
  and c.customer_id <> cr.customer_id -- ← this line can never be true
  or c.first_name <> cr.first_name
  ...
```

Since the `MERGE` already joins `ON c.customer_id = cr.customer_id`, any row reaching `WHEN MATCHED` is guaranteed to already have equal `customer_id` values on both sides. The line `c.customer_id <> cr.customer_id` is logically impossible in this context — it's dead/redundant code (harmless since it's OR'd with real conditions, but contributes nothing). The actual update trigger is the chain of real attribute comparisons (`first_name`, `last_name`, `email`, etc.) — if **any** differ, the row updates in place, overwriting old values (classic Type 1: no history kept).

Setup:

1. The raw `MERGE` statement.
2. Wrapped in a stored procedure `pdr_scd_demo()` that also truncates `customer_raw` afterward.
3. A dedicated `taskadmin` role created and granted `EXECUTE TASK` privilege, then granted to `sysadmin`.
4. A task `tsk_scd_raw` scheduled every 1 minute to call the procedure.

Gotcha: the script ends with `alter task tsk_scd_raw suspend;` — tasks are **suspended by default** and stay that way until explicitly resumed:

```
alter task tsk_scd_raw resume;
```

Step 3.6 — SCD Type 2 (`04_scd_type-2.sql`) — step-by-step execution

This script does five distinct things and is easiest to understand by running it **in stages**, not all at once:

1. **Check the stream first** — `select * from customer_table_changes;` will show 0 rows if nothing has changed in `customer` yet. Expected, not an error.
2. **Make test changes** — the script's `INSERT/UPDATE/DELETE` statements simulate real activity.
3. **Re-check the stream** — now shows rows: an `INSERT` for the new customer, a paired `DELETE+INSERT` for the updated customer (`metadata$isupdate = TRUE`), and a `DELETE` for the removed one.
4. **Create the view `v_customer_change_data`** — this alone moves no data; it's just a saved query translating the stream's raw `metadata$action/metadata$isupdate` columns into clean `dml_type` values ('I', 'U', 'D') for the next step's `MERGE` to use.
5. **Preview the view's output** — `select * from v_customer_change_data;` to sanity-check before wiring it into a task.
6. **Create the task `tsk_scd_hist`** — like `tsk_scd_raw`, this does **not** run immediately; a freshly created task is suspended by default.
7. **Confirm it's suspended** — `show tasks;` shows `state = suspended` for `tsk_scd_hist`. This was the source of "I don't know when this task schedules" confusion — it wasn't scheduled at all yet.
8. **Resume it** — `alter task tsk_scd_hist resume;` (overriding the script's own `alter task tsk_scd_hist suspend;` at the end).

Check the actual schedule:

```
select timestampdiff(second, current_timestamp, scheduled_time) as next_run,
```

```
scheduled_time, current_timestamp, name, state from
table(information_schema.task_history()) where state = 'SCHEDULED' order by completed_time
desc;
```

9. Only after resuming does a row appear here, with `scheduled_time` roughly 60 seconds out.

Verify results — after waiting ~1 minute:

```
select * from customer_history where customer_id in (72, 73, 223136);
select * from customer_history where is_current = true;
select * from customer_history where is_current = false;
```

- 10.

Key mental model — the full chain is NOT one continuous real-time pipeline:

S3 bucket → Snowpipe (auto_ingest, near real-time)

- `customer_raw`
- [Task `tsk_scd_raw`, every 1 min] → customer (SCD Type 1: overwrite in place)
- [Stream `customer_table_changes`, records changes silently]
- [Task `tsk_scd_hist`, every 1 min] → customer_history (SCD Type 2: full history)

Only the first hop (S3 → `customer_raw`) is event-driven/near-instant. Everything downstream runs on independent 1-minute polling schedules — so end-to-end latency from a change happening to it showing in `customer_history` is roughly 1–2 minutes, not instantaneous.

Full Lessons-Learned Summary

1. **Terraform is CLI-only** — no GUI, must be on `PATH`, run from a terminal.
2. **`terraform init` ≠ having a config** — still need `.tf` files before `plan/apply` does anything.
3. **AWS CLI credentials** must match the `profile` referenced in `providers.tf`.
4. **Root credentials work but aren't best practice** — prefer a dedicated IAM user.
5. **Free-tier eligibility changes over time** — verify in the AWS Console, not from memory.
6. **SSH must run from the folder containing the `.pem` key**, or reference its full path — and always from your **local machine**, never accidentally typed inside an already-open remote session.
7. **Windows `.pem` permissions must be locked with `icacls`** — but locking to read-only (`(R)`) will also block you from overwriting the file on a future `terraform apply`; restore to Full Control (`(F)`) first, apply, then lock back to (`R`).

8. **icacIs** syntax differs between PowerShell (`$env:USERNAME`) and `cmd` (`%USERNAME%`) — using the wrong one fails silently with a confusing "invalid parameter" or "syntax incorrect" error.
 9. **A new EC2 instance has nothing installed** — Terraform manages infrastructure only, not application software; Docker/Airflow must be reinstalled and project files re-transferred after every rebuild.
 10. **scp** needs the destination folder to already exist — it won't auto-create nested paths.
 11. **_PIP_ADDITIONAL_REQUIREMENTS** in `.env` requires a container recreate (`down + up`), not just a restart, to take effect.
 12. **Bucket names must match everywhere** — Terraform's bucket, the Airflow DAG's default, and the Snowflake stage URL all need to point at the same S3 bucket, or you'll get silent/confusing `AccessDenied` errors.
 13. **A Snowflake storage integration's `storage_aws_role_arn` must be a real AWS IAM role ARN** — not a Snowflake account identifier or Snowflake role name.
 14. **`file_format = 'CSV'`** (quoted, no parens) looks for a **named file format object**, not a shorthand for "CSV type" — use `file_format = (type = 'CSV' ...)` or a named `file format` object.
 15. **A `MERGE`'s `WHEN MATCHED` condition can contain logically dead comparisons** (e.g. comparing the join key to itself) — harmless if OR'd with real conditions, but worth recognizing as redundant.
 16. **Snowflake tasks are suspended by default** — always `alter task <name> resume;`, or nothing runs on schedule, however correct the SQL is.
 17. **The full SCD pipeline is a chain of independently-scheduled 1-minute tasks**, not one real-time flow — only the S3→Snowpipe hop is close to instant.
-

Cleanup

AWS (stop billing):

```
terraform destroy
```

Snowflake (stop compute):

```
alter task tsk_scd_raw suspend;  
alter task tsk_scd_hist suspend;  
alter warehouse compute_wh suspend;
```

Or drop the whole demo database if fully done:

```
drop database if exists scd_demo;
```